

Software Change in the Solo Iterative Process:

An Experience Report

Christopher Dorman

Department of Computer Science
Wayne State University
Detroit, MI 48202
dorman@wayne.edu

Václav Rajlich

Department of Computer Science
Wayne State University
Detroit, MI 48202
rajlich@wayne.edu

Abstract—This paper reports an experience of a solo programmer who added a new feature into an open source program called muCommander. The process is observed on two granularities: Granularity of software change (SC) and granularity of Solo Iterative Process (SIP). The experience confirms that both SC and SIP process models can be successfully enacted, are able to implement the described feature, and produced a high quality code in reasonable time. The lessons learned, particularly the exit criteria for SC phases, are discussed in more detail in the paper and may be applicable to team iterative processes, including agile processes.

Keywords—agile process; software change; solo programmer; concept location; impact analysis; actualization; refactoring

I. INTRODUCTION

This paper is an experience report of the *Solo Iterative Process (SIP)* as defined in [1, 2]. SIP belongs to the family of iterative processes and it describes a process of a programmer working alone on a software project. It is applicable to the development of small applications that include smartphone applications, to the maintenance of stable products [3], to the development of open source, or as a stepping stone to the teaching of agile processes [2]. SIP shares many characteristics with agile processes that include repeated software change, baseline builds, and so forth.

SIP contains repeated *software change (SC)* [4]. While many authors have addressed the individual phases of SC, an integrated model of SC needs additional exploration. Even though many experienced programmers are capable of executing SC intuitively without the guidance of any model, we want to address less experienced programmers who would benefit from learning SC. We also want to use it to research tools that could aid the SC process. Once the SC process model is established, it can be improved as additional experience is accumulated.

The SC process model consists of several phases and a specific SC enactment may contain all of them or just a subset. The purpose of this study was to explore and gain practical experience with SC within the framework of SIP. Since SC is an indispensable part of both SIP and agile processes, this exploration can provide insight into SC as it applies to agile processes.

This paper presents an experience of using SIP on a medium sized open source program and describes an implementation of a new feature. The feature is implemented

in an iteration that consists of nine SC, each adding new functionality or fixing a bug, and adds 40 new classes to the code that originally contained 1070 classes. This report shows that SIP, including SC, was an effective iterative process, suitable for this software engineering task.

Section II surveys previous work, including the SIP and SC process models. Section III describes the subject program, technologies involved, and the feature to be implemented. Section IV contains the description of the SIP enactment that implements this new feature, with a more detailed description of the first change. Section V contains the discussion of the lessons learned, section VI discusses threats to validity, and section VII contains the conclusions and future work.

II. PREVIOUS WORK

An earlier example of a single programmer process is the Personal Software Process (PSP) [5]. It builds on a programmer's preexisting capabilities and it is taught through a series of programming tasks, where students keep track of a battery of metrics [6]. They learn from their mistakes and that enables them to improve the quality and efficiency of their work. Studies have shown that PSP improves performance in both university and industrial settings [7]. While PSP emphasizes process measurements that are largely independent of the process model, SIP presents a specific process model.

The rest of this section summarizes the two granularities of the process model: the granularity of SC [2, 4] and the granularity of SIP [1, 2].

A. Software Change

SC is a task that is universal to all iterative processes. Researchers have paid a considerable attention to the individual SC phases, but they paid a much smaller attention to the integrated SC model.

The phase of *concept location* finds the specific code snippet that needs to change [8]. Of the numerous concept location techniques and tools that are now available, we used the *dependency search* technique. Concept location by dependency search is guided by *contracts* between suppliers and clients in software [9]. Suppose class A has contracts with its clients that involves concept C. Then concept C is either located in class A, or is delegated to one of its suppliers. This fact can be used in a recursive search for concept C; the process starts at the *main* method or entry point of the program that provides concept C to the user, and

it recursively progresses through client-supplier relationship to the class or method where concept C can be found.

SC often impacts several classes and *impact analysis* determines the strategy and extent of the change. The process of impact analysis uses class interactions that are a superset of the program's dependencies [4]. If a class A changes and it interacts with a class B, then class B may change also; the programmer must inspect it. If class B changes all classes interacting with it can change also and therefore should be inspected. Moreover some classes do not change but propagate the change to the classes interacting with them [4]. The *estimated impact set* is the set of all classes that the programmers predict will change while enacting SC.

Actualization is the phase that actually implements the required new functionality by modifying old classes or creating new ones. The portion of actualization that finds and modifies all the classes that need to change is called *change propagation*. The set of all modified classes is called the *changed set* and should be equal to the estimated impact set.

Refactoring is a phase that changes the structure of software without changing its behavior. It is done either before actualization in preparation for it (prefactoring) or after actualization as a cleaning of the unwanted after effects (postfactoring). There are numerous refactoring transformations and each of them addresses a specific concern, sometimes called an anti-pattern or a bad smell.

Refactoring is a well-researched part of SC [10]. There are a selection of refactorings to choose from; for example, extract class, move field, and others. An updated refactoring list [11] currently has over 90 examples. Mens and Tourwé [12] outline a process for successful refactoring.

Verification assesses and improves the correctness of code updates. Harrold and Soffa looked at the issues of unit testing in evolving software [13]. Zhu, Hall, and May studied techniques to select test criteria [14]. Regression testing verifies whether the old functionality is still functioning after a change. Running the full set of regression tests in large software projects can be very time consuming, in some cases even taking a week or more; therefore much research has been devoted to determining the minimum set of regression tests after a change, such as [15] and [16].

Test-Driven Development (TDD) [17] is a SC process that is based on writing tests first, and then production code that passes the tests. Proponents argue that TDD produces superior code that is free of bloat, and the resulting comprehensive regression test suite encourages refactoring.

B. Team Iterative Processes

SC is the fundamental task of the team iterative processes where repeated changes are grouped into iterations [2]. Among them, Scrum is an agile process that plans for short periods of time, known as sprints [18]. At the end of a sprint the current state of the project is assessed and serves as the basis for the planning of the next sprint. Extreme Programming (XP), has a set of twelve practices [19] that include planning for a short time, pair programming, and so forth. An experience with XP was reported in numerous publications, for example in [20].

Cockburn and Highsmith claim that the dominant factor in the success of agile processes is the quality of the people enacting the process [21]. They present the argument that, "people trump process" (p. 131).

The Rational Unified Process (RUP) is an earlier iterative process oriented towards large projects [22].

C. Solo Iterative Process (SIP)

A SIP iteration is a simplified version of an iteration from the agile processes. During each iteration, the programmer elicits and analyzes the requirements, selects the requirements to be implemented during the iteration (called the *iteration backlog*), then implements these requirements through repeated SC. At the end of the iteration, the programmer thoroughly tests the software and builds a new baseline that is the starting point for the next iteration or for the release to the users [1, 2].

III. PROJECT CHARACTERISTICS

The project reported in this paper is an exploratory case study whose purpose is to observe SC within the context of SIP and find the answers to the following questions:

- Are the SC and SIP models complete? Do they account for programmer's activities and tasks?
- Is the SC model an effective process? Do data in the project indicate that SC supports reasonable productivity and quality?
- Do technologies support the programmer's needs in SC and SIP tasks?
- Are tasks and phases defined sufficiently? Do they give the programmer all necessary guidance?

As is common in exploratory case studies, additional data that may provide additional insights were also recorded.

A. The Object System

In the project, we used the muCommander program; it is an open source, cross platform, advanced file manager program [23]. It expands an operating system's native file manager with several features such as browsing file systems over a variety of connections. The code of muCommander has 76 KLOC and 1,070 classes. It is written entirely in Java. It has a JUnit [24] test suite that includes 441 tests covering 18.1 percent of the statements. Its GUI components use the Swing Java Foundation Classes [25].

B. The Solo Programmer

The solo programmer had university experience in programming, mostly in C++, limited experience in Java programming, limited experience with SIP, and was unfamiliar with the muCommander program. He was charged to implement a new muCommander feature that is described in the next section.

C. New Feature

The new feature request can be found on the muCommander website among the Frequently Asked Questions (FAQ):

"How can I search for a specific file?" The reply: "At the time of writing, you can't. This is an often requested feature,

one that we're thinking about and have a few ideas on how to implement, but it's not there yet.” [23] (pg. FAQ q. 5)

D. Technologies Used in the Project

Besides the technologies of the existing muCommander project, several technologies assisted the programmer in SIP and SC. The programmer used *Subversion (SVN)* as the project version control system [26]. For access to the SVN repository, the programmer used *TortoiseSVN* [27], including its *diff* tool.

The *Clover Java Code Coverage & Test Optimization* tool measures test coverage [28]. *JRipples* is an Eclipse plugin that supports concept location, impact analysis, and actualization [28].

The *Mylyn* task and application lifecycle management framework is included with Eclipse [29]; it assisted the programmer in managing and measuring the effort. To record and export timing data in the minute granularity required an additional plugin called *Tasktop* [30].

The original subject program had no functional tests. The programmer used the *Abbot Java GUI Test Framework* as a technology to build functional tests for the new code [31]. It is based on the JUnit test framework and the Java Virtual Machine automated robot classes. It has classes that help a programmer test many Swing GUI components.

IV. SIP ENACTMENT: ADDING A NEW FEATURE

This section describes an enactment of SIP that implements the new feature. It contains the creation of the iteration backlog, a detailed explanation of the first change, and a description of build at the end of the iteration.

A. Iteration Backlog

The feature request is vague and complex. In order to understand the feature request better, the programmer familiarized himself with the object program; he visited the website and then used the program as his file explorer for two days. This time was not recorded in the timing logs.

TABLE I. ITERATION BACKLOG

#	Title	Change Request
1	Basic Search	Add a basic search function that searches in the current directory for all or part of the title of a folder or file.
2	Recursive Search	Add search inside all directories.
3	Advanced Output	Change the output to a window similar to the main muCommander window.
4	Date Search	Add search by a date of file's modification.
5	Case Sensitive Search	Add search by case sensitive search terms.
6	Extension Search	Add search for files with specific extensions.
7	Properties Search	Add search for files based on their properties.
8	Directory Chooser Bug	File chooser doesn't update the search directory.
9	Date Bug	DateOption is not removed when disabled.

Based on this experience, the programmer divided the feature request into seven change requests of the product backlog in Table I. The last two change requests in Table I address the bugs that the programmer found during the iteration and added them to the backlog later.

B. Change1: Basic Search

This section describes details of change 1 and is an example of SC enactment. The change request is: “Add a basic search function that searches in the current directory for all or part of the title of a folder or file.” During the change request analysis, the programmer added the following further details:

- Activate the search in one of the following ways: the “Go” menu, the quick launch toolbar, and a virtual key.
- Create a search window where the user can enter search terms, start a search, and see the results.
- Write a search algorithm.

1) Concept Location

The programmer extracted the following significant concept: “activate the search”. He started concept location by dependency search by inspecting the `Launcher` class, which contains the program’s main method, see Figure 1. The concept is not located there and JRipples identified 43 suppliers. The programmer inspected `AbstractFile`, `AbstractNotifier`, and `ActionKeymapIO` and concluded that they did not contain the concept. The next class inspected, `ActionManager`, contained a selection of program actions and the programmer realized that this is where the bulk of search functionality would be added. In total, the programmer inspected 5 classes.

2) Impact Analysis

JRipples identified 172 classes that interact with `ActionManager`. The programmer visited 16 of these classes whose names indicated they might have responsibility for muCommander’s toolbar and “Go” menu. Of them, he marked `MainMenuBar`, the class that is responsible for the “Go” menu, as Impacted, and the class `ToolBar` as Propagating.

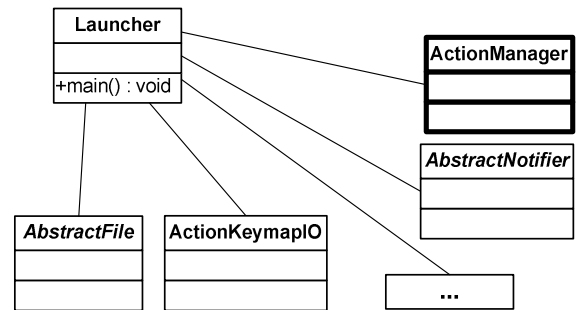


Figure 1. Classes inspected during concept location.

The change propagated through the class `ToolBar` to `ToolBarAttributes` that defines the buttons that select various features and hence it has to change. `ToolBar`

interacts with 24 additional classes and the programmer visited `ToolBarAttributes` first, because its static method `getActions` is used as a parameter for a method call in the `ToolBar` constructor. At this point the programmer was convinced that he understood the scope of the change because he accounted for all the functionality to be added and exited impact analysis. In total, the programmer inspected 16 classes with 2 classes marked Changed and the rest marked Unchanged, see Fig. 2.

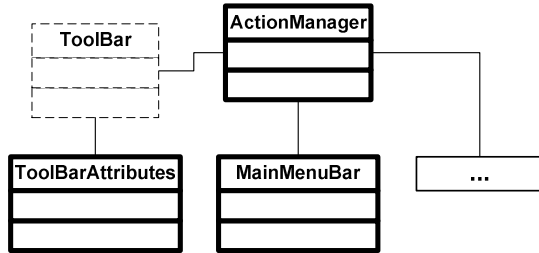


Figure 2. Classes inspected during impact analysis.

3) Actualization and Refactoring

There was no prefactoring in this change. In actualization, the programmer added 2 classes. The first, `SearchAction` was created and then incorporated as a supplier of `ActionManager` through a factory method design pattern [32] that the existing code uses.

The second class is `SearchDialog`, which creates the search window and implements the search algorithm. It is a component of `SearchAction`. To create the class, the programmer copied the code of the existing class `RunDialog` that the programmer visited during impact analysis; the code of this class was thoroughly edited to assume the new responsibility. The original suppliers of `RunDialog` also became suppliers of `SearchAction`.

Once these two classes were incorporated, the programmer implemented the activation functionality in `MainMenuBar` and `ToolBarAttributes`.

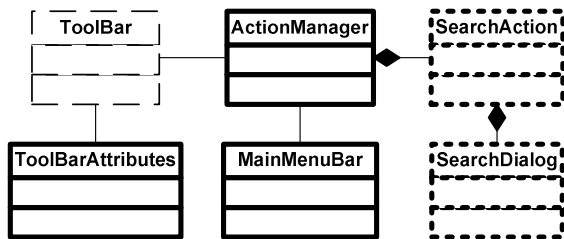


Figure 3. Classes visited during Actualization

Two classes were added to the testing harness; one class for unit testing and one for functional testing. For easy reading, the testing harness is left out of Figs. 1, 2, and 3.

During postfactoring, old comments were deleted and new comments added. The verification of actualization and postfactoring, found no bugs.

C. Remaining Changes and Build

After change 1, the remaining changes of Table I followed. A detailed description can be found in [33]. The UML class diagram of the classes involved in the iteration is in Fig. 4.

At the end of the iteration, the programmer thoroughly tested `muCommander` by running all the regression tests. All tests passed. The programmer then created a baseline without the harness for release to the users. This completed the SIP iteration.

V. OBSERVATIONS

This section answers the questions that were posed as the goal of the cases study.

A. Are the SC and SIP models complete?

SIP worked as defined during the whole iteration; one minor exception is explained in the rest of this section.

In the process model, the refactoring and actualization phases are separated. However in postfactoring of change 3, the programmer extracted the responsibility of stopping the thread that iterates through the file system from the class `SearchDialog` into a new class `SearchThread`. After that, he noticed a short delay of about a second when the “Stop” button is pressed. The programmer used the opportunity and improved the response time and also added a GUI notification that the search is stopped.

According to prescribed practice, the programmer should open a new SC that deals with this new functionality. However the programmer justified this exception because it was small and added only 5 LOC of code. In spite of this experience, the separation of postfactoring that restructures the code and actualization that changes the functionality into two separate phases proved to be useful as it clarifies what activities the programmer should concentrate on.

Exceptions like these are to be expected in all models where there is always tension between the model’s simplicity and accuracy. We concluded that the model presented in this paper is a useful guide to the programmer, but its enactment should allow minor occasional tailoring that corresponds to the project needs.

B. Is the SC model an effective process?

Table II summarizes the numerical data for the phases of the changes 1 through 9. The rest of this section contains a discussion of these data.

1) Productivity

The SIP iteration introduced 4,564 new lines of code in 144 hours and 24 minutes of programmer’s time, which translates to a productivity of 31.6 LOC per hour. For comparison, a previous study followed two groups of PSP students and found the 25-to-75 percentile to produce between 12 and 42 LOC per hour [34]. The productivity of this SIP iteration is roughly in line with this study.

2) *Code Quality*

There were 11 known bugs introduced during the SIP iteration. Of these, 9 were fixed immediately and 2 were added as new change requests to the iteration backlog. No regression bugs were found in the preexisting code in any of the changes and all bugs were introduced in the code added during the iteration, see the data in Table II.

3) *SC Size*

Most SCs took from 13 to 22 hours of work, see Table II. The smaller outliers, changes 8 and 9, turned out to be less productive; they produced 9.6 and 29.3 LOC per hour respectively, less than the project average. This happened in spite of the fact that the programmer did not spend any time performing concept location or impact analysis during these changes.

On the other hand, change 3 was the larger outlier and took almost a week of work. It dealt with GUI components, data structures, sorting algorithms and more. Although its productivity was in line with the average at 31.3 LOC per hour, the programmer struggled with actualization because of its complexity. In hindsight, this change should have been divided into several smaller changes either when creating the iteration backlog, or during impact analysis. During impact analysis, the programmer failed to foresee the full extent of the actualization complexity. In order to make actualization easier, the programmer concentrated on new functionality and sacrificed the structure of the code; he left numerous concepts in improper locations in order to limit change propagation. The programmer then corrected these bad smells during postfactoring, which became large.

This controversial practice of simpler actualization at the expense of more complex postfactoring is available for similarly complex changes.

4) *Amount of Rework*

The new feature has 4,564 new LOC. During the iteration, 1,452 LOC were deleted; these are the lines of either the original code, or the lines that were introduced in the previous changes and later deleted. To calculate rework, we divided the deleted LOC by the added LOC, which resulted in 31.8 percent rework. For comparison, Boehm and Basili found that rework caused purely by bug corrections accounted for 40 percent to 50 percent of a project [35]. This indicates that the amount of rework required in this SIP iteration is not unusual.

C. *Do technologies support the programmer's needs in all SC and SIP tasks?*

Several technologies were used in SIP and a brief overview of the experience with some of them is given here.

1) *JRipples*

JRipples is an open source tool that supports concept location, impact analysis, and the change propagation portion of actualization. It was especially useful during impact analysis, where certain classes of muCommander interact with hundreds of other classes and identifying them can be a tedious and time consuming task. For instance,

during change 7, marking `AbstractFile` as impacted added 307 interacting classes to the inspection list. JRipples has heuristic tools that rank the interacting classes that are most likely to be impacted. However the programmer still had to use judgment to decide when a sufficient impact analysis has been completed.

The programmer also made several specific change requests that would improve functioning of JRipples, and added them to the JRipples website.

2) *Clover Java Code Coverage & Test Optimization*

The programmer used Clover [28] to collect the statement level test coverage. It computed totals and percentages and highlighted the statements already executed. Clover allows creating custom metrics and the programmer created a metrics specifically suitable for the project reports. There were however problems with Clover efficiency.

3) *Mylyn & Tasktop*

The programmer used Mylyn and Tasktop to log timing data for different phases. The timer pauses when the Eclipse window is not the active; for example the programmer could respond to an email without having to manually pause the timer and without corrupting it; the programmer found that very useful.

4) *Abbot Java GUI Test Framework*

The functional tests are written in Abbot in a style similar to JUnit tests. The built in robot test classes are easy to work with and there are specific classes for the Swing library classes. Overall Abbot worked well for the programmer, but he did run into a few issues.

The first issue was that the functional tests run much slower than unit tests. It is in part because Abbot does not support a onetime setup method for an entire test suite; if numerous objects must be created, they must be created for each test in the suite. These issues lead to two change requests, one to do an optimization of Abbot and the second to add the capability for a onetime setup.

A related issue was that the tests were inconsistent, which seemed to be caused by the excessive use of resources. When tests classes were run individually, they would pass without problem. When all the tests in the project were run, at times they would pass and other times they wouldn't. The error message indicated that Abbot couldn't find the GUI component. Rerunning the failed tests was one workaround. Another was to add a delay to the test, but this would slow the test even more.

The last issue was that Abbot was not able to find some modified Swing components. An example of this is the `ComponentTitledBorder` class [36] that adds Swing components to a border. This class did not have a specific Abbot tester and the existing Abbot tester could not find the component in the border. The programmer created a workaround, based on the components coordinates, but they could fail on other computers. The programmer would like more documentation on how to write general custom testers.

5) *Subversion & TortoiseSVN*

Subversion and TortoiseSVN met all the version control needs of the programmer.

6) *DiffStats*

The programmer found a variety of diff tools that could visually show the added, deleted and changed LOC in a single file. However, these tools didn't provide LOC totals for the categories. Therefore the programmer created a tool called DiffStats that calculates added, moved and deleted LOC data. This tool DiffStats can be expanded and refined for future use [37].

D. *Concept Location Experience*

The last question that was posed as a goal of this experience report asks whether the phases of SC are defined sufficiently. The programmer followed the recommendations for the individual phases of SC, and his experience is summarized in the remaining parts of this section V.D through V.I. This particular part deals with Concept Location experience.

The complete process of concept location was performed only during the first and third changes. The other changes used concepts closely related to the previous ones, and the programmer already knew where to find them; the data in Table II correspond to that. Although the time spent during the concept location is a small fraction of the total iteration time, it is the indispensable key that opens the path to the rest of SC and cannot be underestimated.

1) *Exit criteria*

The exit criterion of concept location is well-defined: The concept location ends when the location of the significant concept extension has been found.

E. *Impact Analysis Experience*

Table II shows the estimated impact sets that were produced by impact analysis and the changed sets. The experience indicates that a reasonable goal of impact analysis is to predict prefactoring and actualization only. The modifications during postfactoring are separate, because the postfactoring only loosely belongs to a specific SC and can often be postponed. Also the need for postfactoring becomes obvious only during and after the actualization.

1) *Example of Overestimate: Change 3*

Change 3 included a super class extraction [11] from the class `FileTable`, a large class with many clients and 6 suppliers. The programmer added all of its 6 suppliers to the estimated impact set; however two of the suppliers were not impacted and that led to the overestimate.

2) *Example of Underestimate: Change 7*

In change 7, the programmer missed the impact to one production class. The programmer reported that clients and suppliers to the abstract class `AbstractFile` wouldn't be impacted by the change; `AbstractFile` interacted with 308 classes as identified by JRipples and the programmer did not inspect all of them, because he visited and used

`AbstractFile` in previous changes and became confident that he understood the class and its interactions with other classes. However, during change 7 it became apparent that the code did not work as the programmer believed.

The programmer was unfamiliar with the proxy design pattern [32]. The class `ProxyFile` is a subclass of `AbstractFile` within that pattern and overrides all the abstract methods of `AbstractFile` so that subclasses of `ProxyFile` can override only those methods that are necessary to meet their specific responsibilities. A programmer with knowledge of this design pattern would have visited `ProxyFile` and added it to the estimated impact set.

3) *Harness Code Impact*

In total, the impact of a change on harness code was greater than the impact on production code and was more difficult to predict. The programmer found that harness classes have more class interactions than the classes that they test.

4) *Exit criteria*

A textbook exit criterion would be to visit all classes that interact with the impacted and propagating classes. However in the context of classes that interacts with a large set of other classes, this can be burdensome and time consuming. The programmer accepted the following exit criteria: Whenever the programmer concluded that more than 60 percent of the impacted classes were inspected, he exited the impact analysis phase with the conviction that the scope of prefactoring and actualization is sufficiently well understood and the quality of the SC will not be negatively impacted. An analogy is in testing which also often allows less than 100 percent coverage.

However, change 3 is an example of the importance of understanding not only the extent but also the complexity of an impact. Although the programmer overestimated the impact set, he still underestimated the complexity of it, as already discussed in section V.B.3. The lesson here is that impact analysis must not only predict the impacted classes, but also the complexity and difficulty of their impact.

F. *Prefactoring*

During change 1 the programmer skipped prefactoring. In hindsight, he should have extracted classes for the input and output panels that would have made the change actualization easier. That was later remedied by prefactoring during change 2, but at a higher cost because a larger amount of code had to be moved.

In change 8, the programmer could have skipped prefactoring too, but he extracted a method [11] called `directoryFieldUpdate`. He did this to prepare the code for actualization, which replaced one interface with another. The old interface invoked the method `directoryFieldUpdate` once, but the new interface needed to invoke it from 2 methods. If the programmer had

not done the extract method during prefactoring, he would have had to duplicate code during actualization and then delete it during postfactoring, resulting in more work. Overall, the programmer found that aggressive prefactoring often makes actualization substantially easier.

In some instances, prefactoring involved the introduction of patterns into software, for example a strategy pattern [32] during change 5. The programmer extracted `SearchManager` from `SearchThread` to create the pattern. This made actualization simpler and limited change propagation to one class.

1) Exit criteria

The prefactoring was completed when the local structure of the code was suitable for actualization. In particular, all large significant concepts involved in actualization had a class of their own and for that, some classes were extracted from other classes if necessary. If the planned actualization used polymorphism that does not exist in the old code, a base class was introduced by refactoring. If the planned actualization used a pattern (composite), the pattern was fully prefactored before actualization started.

G. Actualization Experience

During actualization, the programmer implemented the new functionality and incorporated it into the old code, including all secondary changes in affected classes.

1) Exit criteria

The exit criterion for actualization was based on the completeness and quality of implementation of the change request. The programmer determined that all tests (unit, functional and regression) pass. The programmer made sure that each part of the change request is tested, including both valid and invalid inputs, and that the statement coverage of new or modified code is close to 60 percent or more.

H. Postfactoring

Postfactoring involved a clean-up of the consequences of one or several previous changes. For example, the class `InputPanel` was created in change 2 and it added responsibilities during changes 3, 4, and 5, making it large and difficult to understand. In postfactoring after change 5, the programmer solved this accumulated problem by extracting the class `BasicOptionsPanels` from `InputPanel`. It contained the GUI components responsible for the search terms, including case sensitive and recursive search inputs. `InputPanel` was left with the responsibility to assemble the search input panels. After the class extraction both of the classes were responsible for a single significant concept extension, making future changes easier. This gradual build-up of problems in the code after several SC was observed several times and resolved by postfactoring. This build-up belongs to the category of technical debt [38].

1) Exit criteria

Beck and Fowler used vaguely defined “bad smells” as the entry criterion for refactoring and quoted Grandma

Beck, “If it stinks, change it.” [10] (p. 75). The programmer reversed this vague adage into: “When it no longer stinks, stop.” When each new code construct has an identifier that explains its responsibility, all new or modified methods deal with a single responsibility, and all new or modified classes implement a single significant concept, then the postfactoring is done. The programmer used the LOC metric as a guideline to identify suspect code constructs; methods longer than 10 LOC and classes longer than 100 LOC were scrutinized. However the postfactoring was limited to the new or modified code and the programmer did not attempt to refactor the rest of `muCommander`.

1. Frequency of commits

The programmer committed the code to the repository not just at the conclusion of SC, but also after prefactoring and actualization. This is in line with the recommendation of frequent small commits [39].

VI. THREATS TO VALIDITY

This experience report offers a glimpse into a SC and SIP enactment, in the context of one specific programmer, one specific program, one specific feature added, and one specific combination of technologies. We drew some observations from this experience, but transferring them to other contexts should be done with caution.

The program we used may have contributed to the success of this project. It was in an evolvable state [40] and the programmer did not detect any significant code decay that would hinder the evolution.

This project used a variety of tools that also contributed to the success of the iteration. In particular, the tool JRipples contributed significantly to the phase of impact analysis. Identifying class interactions would have been significantly more difficult without it. The other tools were also integral to the success of the SIP iteration.

Finally, the SIP iteration was done in a university setting with a professor and a peer student standing in for users. These users have different motivations than other users of software. While the iteration met the needs of these users, it is possible that it would not meet the needs of other users.

VII. CONCLUSIONS AND FUTURE WORK

This paper reports a case study where the Solo Iterative Process (SIP) was successfully used by a programmer to add a substantial feature to a medium sized open source software. The core of SIP is the task of software change (SC). The case study proved that SIP and SC represent an operational model and based on this experience, it is possible to recommend this model for similar tasks. SC is an integral part of a variety of iterative software team processes, including the agile processes. We speculate that the presented model of SC is usable within these processes also; however further study of SC in the context of these team processes may shed additional light and is the next logical research step.

The reported experience deals with the individual phases of SC. We found that impact analysis played an important role in software change planning, and could predict the extent of prefactoring and actualization of SC. However it could not predict postfactoring that cleans the code after the change. Postfactoring was done reactively, based on the code properties immediately after actualization, and sometimes resolved gradual accumulation of the code problems that were left unresolved in several previous changes. The programmer made a decision whether to address these problems immediately, or whether to postpone them further, and that may explain why this phase was difficult to predict during impact analysis. We speculate that this is a common problem of iterative and agile processes.

In this project, we required the programmer during impact analysis to inspect approximately 60 percent of the classes that were to be modified in SC and found that criterion to be sufficient for our specific productivity and quality goals. However more accurate exit criteria for impact analysis are a topic to be explored in the future; the heuristics available in the literature still need additional research before they will be ready for practical use [41].

Another important lesson learned is that the impact analysis phase needs to predict not only the set of impacted classes, but also the complexity of their impact. That complexity can play a role in determining the strategy of SC; if it exceeds a certain limit, it may be preferable to split the SC into several ones. One of the topics for future work is to quantify the impact on the classes. Currently, researchers and practitioners use a binary “impacted – not impacted” scale, but a wider range may more accurately predict the difficulty of the SC and help avoid surprises during the actualization phase.

The programmer used several diverse technologies to support SC that significantly contributed to the success of the project. The strengths and weaknesses of these technologies were assessed in the case study, and it became clear that a lack of their integration was a hindrance. We speculate that technologies have a similar impact on SC within agile and iterative team processes. A unified and seamless software environment that would integrate them and support all SC needs is another topic for future work.

ACKNOWLEDGMENTS

We would like to thank to Radu Vanciu for his help with this project. We would also like to thank Atlassian Pty Ltd and Tasktop Technologies who provided their proprietary tools, Clover Java Code Coverage & Test Optimization and Tasktop, free of charge for use in this project. This work was supported in part by grant from the National Science Foundation CCF-0820133. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the NSF.

REFERENCES

- [1] N. Febraro, Rajlich, V., "The Role of Incremental Change in Agile Software Processes," in *Agile 2007*, pp. 92-103.
- [2] V. Rajlich, *Software Engineering: The Contemporary Practice*. Boca Raton, FL: CRC Press, 2012.
- [3] A. J. Rostkowycz, V. Rajlich, and A. Marcus, "A case study on the long-term effects of software redocumentation," in *IEEE International Conference on Software Maintenance*, 2004, pp. 92-101.
- [4] V. Rajlich and P. Gosavi, "Incremental change in object-oriented programming," *Software, IEEE*, vol. 21, pp. 62-69, July/August 2004.
- [5] W. Humphrey, *Introduction to the personal software process*. Reading, MA, USA: Addison-Wesley, 1996.
- [6] W. S. Humphrey, "Using a defined and measured personal software process," *Software, IEEE*, vol. 13, pp. 77-88, May 1996.
- [7] P. Ferguson, W. S. Humphrey, S. Khajenoori, S. Macke, and A. Matvya, "Results of applying the personal software process," *Computer, IEEE*, vol. 30, pp. 24-31, May 1997.
- [8] B. Dit, M. Revelle, M. Gethers, and D. Poshvanyk, "Feature location in source code: a taxonomy and survey," *Journal of Software: Evolution and Process*, p. to be published, 2012.
- [9] K. Chen and V. Rajlich, "Case study of feature location using dependence graph," in *IEEE 8th International Workshop on Program Comprehension*, Limerick, Ireland, 2000, pp. 241-247.
- [10] M. Fowler, K. Beck, J. Brant, W. Opdyke, and D. R. Roberts, *Improving the design of existing code*: Addison-Wesley, 1999.
- [11] M. Fowler. (2010, June 6, 2011). *Refactoring*. Available: <http://refactoring.com/>
- [12] T. Mens and T. Tourwé, "A survey of software refactoring," *Software Engineering, IEEE Transactions on*, vol. 30, pp. 126-139, Feb 2004.
- [13] M. J. Harrold and M. Souffa, "An incremental approach to unit testing during maintenance," in *IEEE Conference on Software Maintenance*, 1988, pp. 362-367.
- [14] H. Zhu, P. A. V. Hall, and J. H. R. May, "Software unit test coverage and adequacy," *ACM Computing Surveys (CSUR)*, vol. 29, pp. 366-427, December 1997.
- [15] H. Agrawal, J. R. Horgan, E. W. Krauser, and S. A. London, "Incremental regression testing," in *IEEE Conference on Software Maintenance*, 1993, pp. 348-357.
- [16] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold, "Prioritizing test cases for regression testing," *IEEE Transactions on Software Engineering*, vol. 27, pp. 929-948, October 2001.
- [17] R. C. Martin, "Professionalism and test-driven development," *IEEE Software*, vol. 24, pp. 32-36, May/June 2007.
- [18] K. Schwaber and M. Beedle, *Agile software development with Scrum*. Prentice Hall, 2002.
- [19] K. Beck, *Extreme programming explained: embrace change*. Boston: Addison-Wesley Professional, 2000.
- [20] M. M. Müller and W. F. Tichy, "Case study: extreme programming in a university environment," in *23rd International Conference on Software Engineering*, Toronto, Ontario, Canada, 2001, pp. 537-544.
- [21] A. Cockburn and J. Highsmith, "Agile software development, the people factor," *Computer, IEEE*, vol. 34, pp. 131-133, November 2001.

- [22] P. Krutchen, *Rational Unified Process: an introduction*: Addison-Wesley Professional, 2004.
- [23] M. Bernard, *MuCommander*. Available: <http://www.mucommander.com/> (2011).
- [24] K. Beck, *JUnit*. Available: <http://JUnit.org> (2011).
- [25] Oracle, *Swing (Java Foundation Classes)*. Available: <http://download.oracle.com/javase/6/docs/technotes/guides/swing/> (2011).
- [26] Apache Software Foundation, *Subversion*. Available: <http://subversion.apache.org/> (2011).
- [27] The TortoiseSVN Team, *TortoiseSVN*. Available: <http://tortoisesvn.net/> (2011).
- [28] Atlassian Pty Ltd., *Clover Java Code Coverage & Test Optimization*. Available: <http://www.atlassian.com/software/clover/> (2011).
- [29] The Eclipse Foundation, *Mylyn*. Available: <http://www.eclipse.org/mylyn/> (2011).
- [30] Tasktop Technologies, *Tasktop*. Available: <http://tasktop.com/> (2011).
- [31] T. Wall, *Abbot Java GUI Test Framework*. Available: <http://abbot.sourceforge.net/doc/overview.shtml> (2008).
- [32] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design patterns: elements of reusable object-oriented software* vol. 206. Reading, MA: Addison-wesley 1995.
- [33] C. Dorman, "An Experience Report Of The Solo Iterative Process," M.S., Computer Science, Wayne State University, Detroit, MI, 2011.
- [34] L. Prechelt and B. Unger, "An experiment measuring the effects of personal software process (PSP) training," *Software Engineering, IEEE Transactions on*, vol. 27, pp. 465-472, May 2001.
- [35] B. Boehm and V. R. Basili, "Software defect reduction top 10 list," *Computer, IEEE*, vol. 34, pp. 135-137, January 2001.
- [36] S. Kumar, *ComponentTitledBorder*. Available: http://www.jroller.com/santhosh/entry/component_titledBorder (2005).
- [37] C. Dorman. (2011, Apr. 4). *DiffStats*. Available: <http://sourceforge.net/projects/diffstats/>
- [38] N. Brown, Y. Cai, Y. Guo, R. Kazman, M. Kim, P. Kruchten, E. Lim, A. MacCormack, R. Nord, and I. Ozkaya, "Managing technical debt in software-reliant systems," in *SE/SDP workshop on Future of software engineering research*, Santa Fe, New Mexico, USA, 2010, pp. 47-52.
- [39] P. M. Duval, *Continuous integration*: Addison Wesley, 2007.
- [40] T. Tamai and Y. Torimitsu, "Software lifetime and its evolution process over generations," in *IEEE Conference on Software Maintenance*, 1992, pp. 63-69.
- [41] S. Hassaine, F. Boughanmi, Y. G. Guéhenec, S. Hamel, and G. Antoniol, "A Seismology-inspired Approach to Study Change Propagation," in *IEEE International Conference on Software Maintenance* 2011, pp. 53 - 62.

TABLE II. SIP DATA BY CHANGE

Phase	Change									
	1	2	3	4	5	6	7	8	9	Total/AVG
Concept Location	0:22	0:00	0:33	0:00	0:00	0:00	0:00	0:00	0:00	0:55
Impact Analysis	2:08	2:28	3:23	1:26	1:02	0:55	0:38	0:00	0:00	12:00
Prefactoring	0:00	1:22	2:11	1:41	9:32	3:06	0:00	0:07	0:00	17:59
Prefactoring Testing	0:00	2:43	0:07	0:41	2:53	0:55	0:00	0:09	0:00	7:28
Actualization	5:34	3:41	4:08	4:42	1:36	2:20	2:57	0:16	0:23	25:37
Actualization Testing	5:02	1:52	6:42	3:34	0:49	2:36	2:32	0:37	0:22	24:06
Postfactoring	0:23	2:57	15:49	4:46	2:35	3:18	3:54	0:00	0:00	33:42
Postfactoring Testing	0:12	7:34	5:34	1:28	1:19	2:08	4:22	0:00	0:00	22:37
Total (Hours:minutes)	13:41	22:37	38:27	18:18	19:46	15:18	14:23	1:09	0:45	144:24
Classes										
Estimated Impact Set – Production Code	3	1	14	6	6	6	3	1	3	43
Changed Set - Production Code	3	1	8	6	6	3	4	1	3	35
Changed in Postfactoring - Production Code	0	0	11	0	0	3	1	0	0	15
New classes added - Production Code	2	4	9	9	7	1*	8	0	0	40
Estimated Impact Set – Harness	0	2	7	7	10	5	4	2	4	41
Changed Set – Harness	1	3	3	6	9	3	3	2	3	33
Changed in Postfactoring - Harness	0	0	4	0	1	2	8	0	0	15
New classes added - Harness	2	5	7	12	6	3*	9	0	0	44
Tests										
Bugs in Changed Classes	0	3	2	3	1	2	0	0	0	11
Bugs fixed when found	0	3	2	2	0	2	0	0	0	9
Bugs Postponed	0	0	0	1	1	0	0	0	0	2
New assertions added	11	61	125	239	79	99	107	3	7	731
New statement coverage	67.7%	77.5%	59.7%	90.8%	94.5%	96.0%	84.8%	98.2%	96.7%	74.0%

* Change 6 added 5 new production classes and deleted 4, and added 4 new harness classes and deleted 1

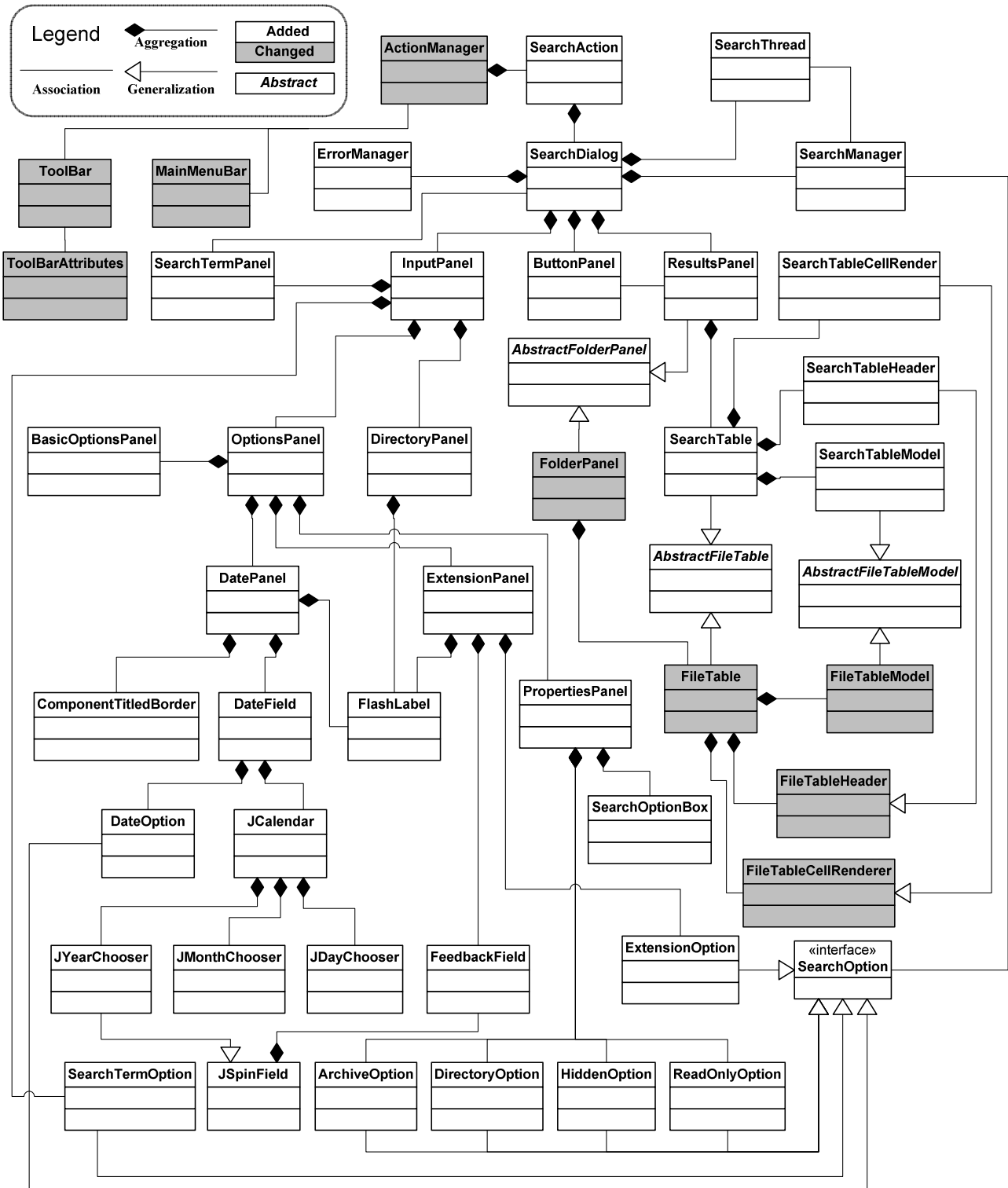


Figure 4. Classes involved in SIP iteration